

Relatório de Comparação de Algoritmos de Busca

Busca sequencial, busca jump e busca binária em conjuntos de dados de 50 a 250.000 elementos

Objetivo. Avaliar como o crescimento de N afeta o tempo de execução e o número de comparações, relacionando os resultados à necessidade de recuperação eficiente de informação em cenários de Big Data.

Conclusão principal. Nos testes fornecidos, a busca binária foi a alternativa mais estável para listas ordenadas. Em N = 250.000, ela usou 17 comparações e 0,000107 s, contra 125.001 comparações e 0,317401 s da busca sequencial.

Escopo e método

Foram analisados 28 casos de teste (84 medições): três algoritmos para cada tamanho de array. O alvo de cada caso foi escolhido a partir do elemento central do array ordenado. A medição de busca foi isolada com timeit; o tempo de ordenação não foi incluído nos resultados recebidos. A memória de pico reportada foi 39,65 KB em todas as medições.

Fundamentação e Big Data

Na hierarquia DIKW, dados brutos precisam ser recuperados e organizados para se converterem em informação útil e, depois, conhecimento. À medida que o volume aumenta, um algoritmo de busca ineficiente transforma a recuperação em gargalo. Por isso, comparar custo linear, aproximadamente raiz quadrada e logarítmico é relevante para entender a relação entre volume e velocidade.

Complexidade teórica

Algoritmo	Pré-requisito	Complexidade	Comportamento esperado
Busca sequencial	Nenhum	$O(n)$	Crescimento proporcional ao número de elementos
Busca jump	Lista ordenada	$O(\sqrt{n})$	Salto por blocos e varredura local
Busca binária	Lista ordenada	$O(\log n)$	Elimina metade do espaço a cada comparação

Resultados empíricos

Os gráficos utilizam escala logarítmica nos dois eixos para tornar comparáveis valores muito pequenos de tempo e grandes diferenças de esforço. A tendência observada é coerente com as complexidades teóricas, embora tempos absolutos variem por hardware, cache e carga do ambiente.

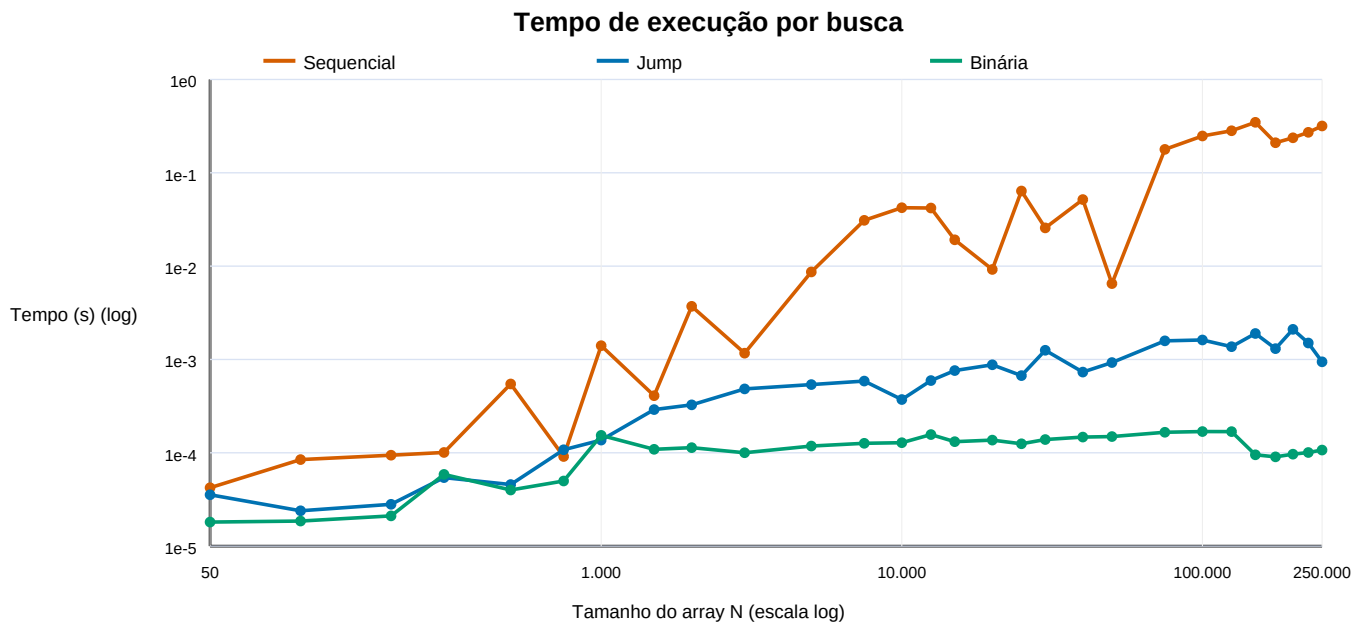


Figura 1 - Tempo de execução de uma busca por tamanho do array.

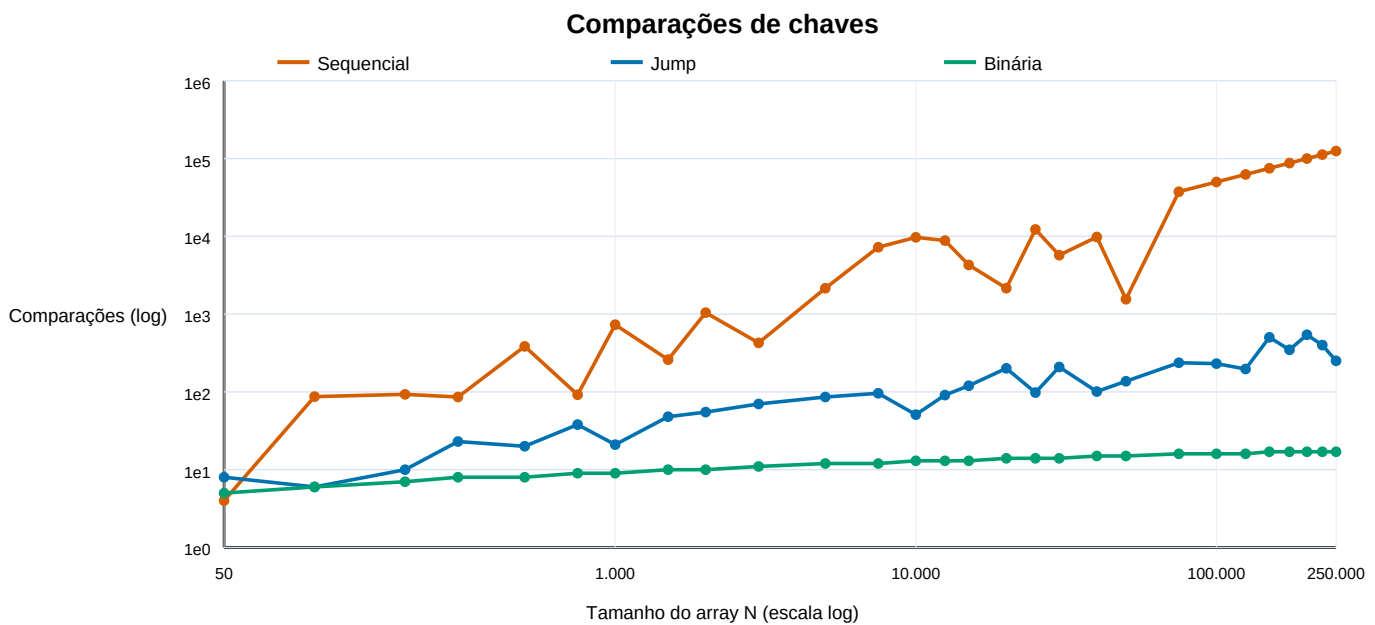


Figura 2 - Comparações de chaves por tamanho do array.

Tabela comparativa de pontos representativos

Cada célula apresenta tempo e, abaixo, número de comparações. A busca sequencial não recebe lista ordenada; jump e binária recebem uma cópia ordenada.

N	Sequencial Tempo / Comp.	Jump Tempo / Comp.	Binária Tempo / Comp.
50	0.000042 s 4	0.000036 s 8	0.000018 s 5
1.000	0.001405 s 732	0.000137 s 21	0.000154 s 9
10.000	0.042255 s 9.716	0.000372 s 51	0.000128 s 13
100.000	0.247963 s 50.001	0.001617 s 231	0.000169 s 16
250.000	0.317401 s 125.001	0.000943 s 251	0.000107 s 17

Interpretação dos resultados

A busca sequencial depende da posição real do alvo no array original. Por isso suas medições não crescem de modo perfeitamente monotônico: em alguns casos o alvo aparece cedo, em outros tarde. Ainda assim, nos maiores conjuntos ela alcança dezenas ou centenas de milhares de comparações.

A busca jump reduz a busca a saltos de tamanho $\sqrt{N}$ e depois examina apenas um bloco. Ela supera a sequencial em conjuntos grandes, mas continua fazendo centenas de comparações: em $N = 250.000$ foram 251.

A busca binária mantém um total muito baixo de comparações porque descarta metade do intervalo a cada passo. Entre $N = 50$ e $N = 250.000$, passou de 5 para apenas 17 comparações. Este é o comportamento que torna o método adequado a consultas repetidas sobre dados já ordenados.

Memória física e ponto de ruptura

Os resultados fornecidos registram 39,65 KB de pico para a rotina monitorada em todos os casos até $N = 250.000$. Não há execução para N acima disso nem registro de falha por RAM; portanto, não é possível afirmar um ponto de ruptura sem fabricar evidência. Para documentá-lo corretamente no Colab, deve-se aumentar N gradualmente, registrar a última execução bem-sucedida, a primeira falha e a mensagem de erro, distinguindo memória do array, da cópia ordenada e do processo Python.

Custo oculto da ordenação

Busca jump e busca binária exigem ordenar o array. Esse pré-processamento tem custo aproximado $O(N \log N)$ e não foi medido em Resultados.txt. Assim, os tempos exibidos comprovam o custo da busca, mas não permitem calcular empiricamente o ponto exato em que ordenar compensa para uma única consulta.

Cenário	Custo aproximado	Leitura prática
Uma única busca	$O(N \log N) + O(\log N)$	A ordenação pode dominar; compare com uma busca sequencial antes de escolher.
10.000 buscas no mesmo array	$O(N \log N) + 10.000 O(\log N)$	A ordenação é amortizada e a binária tende a compensar fortemente.
Limiar de benefício	$q \approx T_{\text{sort}} / (T_{\text{seq}} - T_{\text{bin}})$	Meça T_{sort} e o tempo médio de cada método com várias repetições para obter q .

Como obter o limiar. Cronometre `np.sort(arr)` separadamente uma vez. Depois, para um conjunto de consultas, use $q = T_{\text{sort}} / (T_{\text{sequencial}} - T_{\text{binária}})$. Se o denominador for positivo, q informa aproximadamente quantas consultas são necessárias para recuperar o investimento da ordenação. Para 10.000 consultas, compare os dois custos totais, sem incluir a ordenação dentro de cada busca.

Como o código funciona

1. Os arquivos de teste são extraídos e lidos para um array NumPy. 2. O array é ordenado com `np.sort` somente para jump e binária. 3. Cada algoritmo retorna o índice do alvo e o contador `num_comp`. 4. A função `rodar_teste` mede uma chamada com `timeit`, coleta memória com `tracemalloc` e agrega os resultados. 5. Ao final, um `DataFrame` permite exportar ou gerar gráficos.

Observação de correção

No arquivo entregue há um erro de sintaxe na primeira função: `num_comp = 0a`. A linha deve ser `num_comp = 0`. Sem essa correção, o notebook não executa. O restante da lógica está alinhado ao objetivo do experimento.

Código fonte analisado

Transcrição do arquivo codigo.txt. A única marcação entre colchetes identifica a correção sintática necessária.

```
from google.colab import files
uploaded = files.upload()

"""
Trabalho de Big Data - Comparacao de Algoritmos de Busca
Busca Sequencial | Busca Jump | Busca Binária
Codigo simples para rodar no Google Colab
"""

import math
import time
import timeit
import tracemalloc
import numpy as np

# -----
# 1. BUSCA SEQUENCIAL - O(n)
# -----
def busca_sequencial(arr, alvo):
    num_comp = 0 # corrigido: o original continha "0a"
    for i in range(len(arr)):
        num_comp += 1
        if arr[i] == alvo:
            return i, num_comp
    return -1, num_comp

# -----
# 2. BUSCA JUMP (Salto) - O(sqrt(n)) -- exige lista ORDENADA
# -----
def busca_jump(arr, alvo):
    n = len(arr)
    passo = int(math.sqrt(n))
    num_comp = 0

    inicio = 0
    fim = passo
    # Encontra o bloco onde o alvo pode estar
    while fim < n and arr[min(fim, n) - 1] < alvo:
        num_comp += 1
        inicio = fim
        fim += passo
    fim = min(fim, n)

    # Busca sequencial dentro do bloco
    for i in range(inicio, fim):
        num_comp += 1
        if arr[i] == alvo:
            return i, num_comp

    return -1, num_comp

# -----
# 3. BUSCA BINARIA - O(log n) -- exige lista ORDENADA
# -----
def busca_binaria(arr, alvo):
    baixo, alto = 0, len(arr) - 1
    num_comp = 0

    while baixo <= alto:
        num_comp += 1
        meio = (baixo + alto) // 2
        if arr[meio] == alvo:
            return meio, num_comp
        elif arr[meio] < alvo:
            baixo = meio + 1
        else:
            alto = meio - 1

    return -1, num_comp

# -----
# FUNCAO AUXILIAR: roda um algoritmo medindo tempo, comparacoes e memoria
# -----
def rodar_teste(nome_algoritmo, funcao_busca, arr, alvo):
    print("-" * 50)
    print(f"Metodo: {nome_algoritmo}")

    tracemalloc.start()

    # timeit mede so a funcao de busca (varias repeticoes p/ precisao)
    tempo = timeit.timeit(lambda: funcao_busca(arr, alvo), number=1)

    # roda uma vez "de verdade" para pegar indice e num_comp
    indice, num_comp = funcao_busca(arr, alvo)

    memoria_atual, memoria_pico = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    print(f"Total de elementos (N): {len(arr)}")
    print(f"Indice Encontrado: {indice} (-1 significa que nao existe)")
    print(f"Tempo de execucao: {tempo:.6f}s | Comparacoes realizadas: {num_comp}")
    print(f"Memoria usada (pico): {memoria_pico / 1024:.2f} KB")
    return {
```

```

        "algoritmo": nome_algoritmo,
        "N": len(arr),
        "tempo": tempo,
        "comparacoes": num_comp,
        "memoria_kb": memoria_pico / 1024,
        "indice": indice,
    }

# -----
# LEITURA DOS CASOS DE TESTE (arquivos N.txt dentro do zip)
# -----
import os
import glob
import zipfile

PASTA_ZIP = "/content/Casos_de_Teste.zip" # suba o zip para o Colab com este nome
PASTA_EXTRAIDA = "/content/Casos_de_Teste"

def preparar_arquivos():
    # Extraí o zip (so na primeira vez)
    if not os.path.exists(PASTA_EXTRAIDA):
        with zipfile.ZipFile(PASTA_ZIP, "r") as z:
            z.extractall("/content")

    # Procura a pasta "Construir" dentro do zip, onde estao os N.txt
    caminhos = glob.glob(f"{PASTA_EXTRAIDA}/**/*.txt", recursive=True)
    return sorted(caminhos, key=lambda p: int(os.path.basename(p).replace(".txt", "")))

def ler_array(caminho):
    with open(caminho) as f:
        numeros = f.read().split()
    return np.array([int(x) for x in numeros])

# -----
# EXECUCAO PARA TODOS OS CASOS DE TESTE
# -----
if __name__ == "__main__":

    arquivos = preparar_arquivos()
    print(f"Encontrados {len(arquivos)} casos de teste.\n")

    resultados = []

    for caminho in arquivos:
        arr = ler_array(caminho)
        N = len(arr)

        # Alvo de teste: pega um numero que existe no array (ex: o do meio)
        arr_ordenado = np.sort(arr)
        alvo = arr_ordenado[N // 2]

        print("=" * 60)
        print(f"Arquivo: {os.path.basename(caminho)} | N = {N} | Alvo = {alvo}")

        resultados.append(rodar_teste("Busca Sequencial", busca_sequencial, arr, alvo))
        resultados.append(rodar_teste("Busca Jump", busca_jump, arr_ordenado, alvo))
        resultados.append(rodar_teste("Busca Binaria", busca_binaria, arr_ordenado, alvo))

    print("\n" + "=" * 60)
    print("RESUMO GERAL")
    print("=" * 60)
    for r in resultados:
        print(r)

    # ---- Opcional: transformar em tabela para exportar/plotar ----
    import pandas as pd
    df = pd.DataFrame(resultados)
    print("\nTabela final:")
    print(df)
    # df.to_csv("resultados.csv", index=False) # descomente para salvar

```

Conclusão

A evidência empírica confirma que a busca binária reduz drasticamente o esforço por consulta em listas ordenadas. A busca jump é uma alternativa intermediária, enquanto a sequencial permanece apropriada quando não há ordenação disponível ou a busca é única e o custo de pré-processamento não se justifica. Para atender integralmente à proposta, a próxima execução deve acrescentar a medição de np.sort, 10.000 consultas e o registro real do limite de memória do Colab.